SUP'COM
Higher School of Communication of Tunis

COSIM
Communications, Signal and Image

Engineering internship

Cell image segmentation with deep learning

Achieved by:
Mohamed Karaa

Supervisors:
Amel Benazza
Rabaa Youssef

2020 - 2021

Contents

Acknowledgement

1 Introduction **1**
1.1 Context . . . 1
1.2 Host establishment . . . 1

2 State of the art **2**
2.1 Image segmentation . . . 2
2.2 Cell image segmentation . . . 3

3 EVICAN dataset **4**
3.1 Generalities . . . 4
3.2 Acquisition and validation pipeline . . . 4
3.3 Custom dataset for colon cells . . . 5
3.4 Difficulty levels . . . 6

4 Modeling **8**
4.1 Mask RCNN . . . 8
4.1.1 Overview . . . 8
4.1.2 Architecture blocks . . . 9
4.1.3 Key hyperparameters . . . 10
4.1.4 Loss function . . . 11
4.2 Transfer learning . . . 12
4.3 Evaluation Metrics . . . 12
4.3.1 Jaccard index . . . 13
4.3.2 Precision and recall . . . 13
4.3.3 Mean average precision . . . 13
4.3.4 Metric interpretation . . . 14

5 Training environment **15**

6 Training and Results **16**
6.1 Base model . . . 16
6.1.1 Training strategy . . . 16
6.1.2 Results . . . 17
6.1.3 Visualization . . . 18
6.2 Hyperparameter tuning . . . 19
6.2.1 Tuning strategy . . . 19
6.2.2 Results . . . 21
6.2.3 Visualization . . . 21
6.3 Image augmentation . . . 22
6.3.1 Techniques . . . 22
6.3.2 Results . . . 23

7 Conclusion **25**

List of Figures

1 Difference between image classification, object detection and image segmentation [1]. . . . 2
2 Difference between semantic segmentation and instance segmentation [2]. . . . 3
3 EVICAN dataset construction. . . . 5
4 Dataset samples. . . . 5
5 Colon cell dataset image and instance distribution. . . . 6
6 Difficulty levels samples. . . . 7
7 Mask RCNN architecture overview. . . . 9
8 ResNet-50 architecture. [3] . . . 9
9 ResNet-50 architecture. [4] . . . 10
10 Illustration of Jaccard index. . . . 13
11 Base model learning curve. . . . 17
12 Region proposal network learning curve. . . . 17
13 Visualization of random segmented samples. . . . 18
14 Tuned model learning curve. . . . 19
15 RPN anchors = [8, 16, 32, 64, 128]. . . . 20
16 RPN anchors = [16, 32, 64, 128, 256]. . . . 20
17 Visualization of random segmented samples. . . . 22
18 Comparison between original samples (left) and CLAHE enhancement (right). . . 23

List of Tables

1 Evaluation dataset difficulty criteria. 6
2 List of Mask RCNN key hyperparameters. 11
3 Default hyperparameters. 16
4 Base model results. 17
5 Tuned hyperparameters. 19
6 Optimized model results. 21
7 Model results with image augmentation. 23

Acknowledgement

I want to express my warmest gratitude to my supervisors Professor **Amel Benazza** and Professor **Rabaa Youssef** who have given me the opportunity to undertake this enriching experience within the COSIM lab. I also want to thank them for sharing their time, knowledge and experience, which lead me to realize this project.

Also, I want to thank my family and friends who encouraged me to carry on working on the project and going forward in my learning journey, especially in these hard times.

1 Introduction

1.1 Context

With the ongoing rise of computer science, artificial intelligence, and imaging techniques, biological studies have seen new approaches and tools that help interpret and explain huge amounts of biological data in the hands of experts. This development in the field is now serving the human race as it facilitates doctors and scientists to diagnose illnesses in early stages and develop cures.

The engineering internship, falls within this context. Cell image segmentation with deep learning is a challenging task that aims to locate cells within human tissue, classify them, and delineate the boundaries of these cells.

One primary application of cell segmentation is identifying cancerous cells in a given image. The main objective of this project is to detect and segment cancerous cells in images taken from different colon tissues using a deep learning model. Having an adequate set of images is the key for a deep learning model to succeed in a task. Therefore, the flow of the project begins by studying a specific dataset for cell images named EVICAN. Later on, this dataset is leveraged to train and optimize a deep computer vision model, Mask RCNN. Finally, the model will be evaluated according to defined metrics and objectives.

1.2 Host establishment

The internship was hosted by COSIM (Communication, Signal et Image) laboratory. COSIM is a research unit within the Higher School of Communication of Tunis (Sup'Com). COSIM makes innovative contributions in the field of Information and Communication Sciences and Technologies. Based on their knowledge and competence, COSIM research members lead to implementing solutions that address precise needs in the following fields: health, environment protection, 5G networks, and the Internet of Things.

2 State of the art

2.1 Image segmentation

In computer vision applications, image segmentation is the task of assigning every pixel in an image to a specific class. This technique delivers each object in an image a pixel-wise mask that gives information about the object's coordinates (object detection) and shape.

Image segmentation is a widely used technique in many fields. It is applied in autonomous vehicles to distinguish pedestrians, traffic signs, and other vehicles. Also, image segmentation is used in bio-medicine to detect cancerous cells and tumors.
Image segmentation can be seen as an extension of other computer vision tasks: object detection and classification. In classification, an object is assigned to a specific class regardless of its position in the image. In object detection, different objects are detected in the image, localized with a bounding box, and assigned to a class. The bounding box is the minimal box containing all pixels of an object, but it does not give information about the shape.

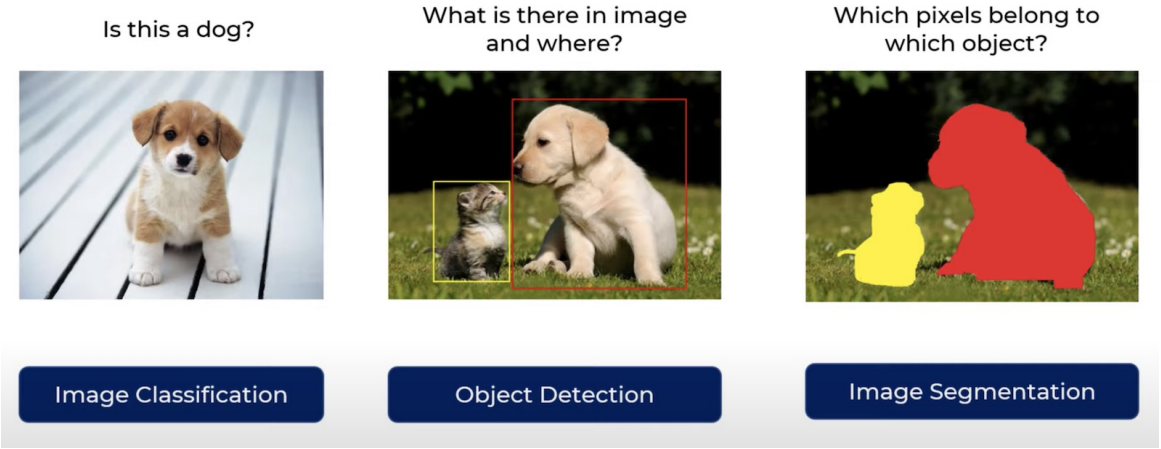


Figure 1: Difference between image classification, object detection and image segmentation [1].

Image segmentation can also be brought down to two different sub-techniques :

- **Semantic segmentation:** Pixels of instances (objects) of the same class are assigned one mask.
- **Instance segmentation:** Pixels of each instance of a class are assigned a different mask from the others.

Figure 2 shows the difference: Semantic segmentation assigns all persons to one class with one color for the segmentation, whereas instance segmentation differentiates each person with different colors.

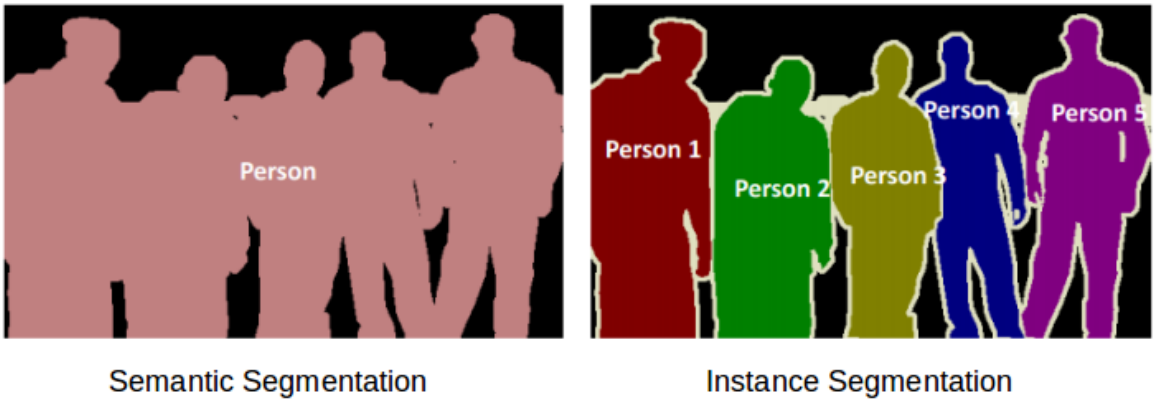


Figure 2: Difference between semantic segmentation and instance segmentation [2].

To achieve such tasks, different methods have been developed with classical and deep approaches. For example and not limited to [5] :

- **Thresholding segmentation:** The simplest method for segmentation in image processing is the thresholding method. It separates pixels in a picture by comparing the intensity of each pixel to a predetermined value (threshold). It comes in handy when the desired object is brighter than the background (unnecessary parts).
- **Clustering-based segmentation:** A clustering technique divides the image into clusters (disjoint groups) of pixels with comparable properties. It would divide the data elements into clusters, where the elements in a cluster are more similar than the elements present in other clusters. One of the used clustering algorithms in image segmentation is k-means clustering.
- **Neural networks for segmentation** : Deep neural networks are also efficient in image segmentation, particularly when leveraging massive datasets. Multiple architectures are employed for both types of segmentation tasks. For example, Mask RCNN for instance segmentation, U-NET for semantic segmentation, and Fully Convolutional Network for semantic segmentation.

2.2 Cell image segmentation

Cell segmentation is the task of splitting a microscopic image domain into segments, which represent individual instances of cells. It is a fundamental step in many biomedical studies, and it is regarded as a cornerstone of image-based cellular research. Well-segmented images can capture biologically relevant morphological information, such as anomalies and illness.

Earlier mentioned segmentation techniques were leveraged to develop cell image applications. Some of the previous works in the field consist of adapting existing architectures to cell images, for example Cell Detection and Segmentation in Microscopy Images with Improved Mask R-CNN [6] that integrates a new method for calculating the loss function. Also, Microscopy cell nuclei segmentation with enhanced U-Net [7] for cell semantic segmentation.

3 EVICAN dataset

3.1 Generalities

EVICAN (Expert visual cell annotation) dataset is a balanced dataset for algorithm development in cell and nucleus segmentation, especially for deep learning applications. It assembles more than 4600 images of 30 different human cell lines. The dataset counts almost 53000 segmented instances, with more than 1000 instances for the majority of categories.
Images in the dataset are grayscale and were acquired using different microscopes (4 microscopes, nine objectives) and different contrast mechanisms (brightfield and phase-contrast).

EVICAN comes in COCO (Common Objects in Context) format annotation. COCO is a large-scale dataset containing over 1.5 million object instances from 80 different object classes used for object detection and segmentation tasks. COCO annotation has a specific format stored in a JSON file with different keys: images, annotations, and category. These keys hold all the information about annotated objects in a dataset.

The dataset is split into :

- Training dataset: 4464 images, partially annotated (not all cell instances are segmented and annotated in the image).
- validation dataset: 1176 images, partially annotated.
- Evaluation dataset: 98 images, divided into 3 difficulty levels and fully annotated

3.2 Acquisition and validation pipeline

EVICAN images were acquired using four different microscopes and two contrast mechanisms to provide a heterogeneous dataset in terms of instrument variation. It also contains 30 different classes of cells and nuclei, i.e., 60 object classes, providing a balanced dataset.
Images acquired by microscopes are later annotated by cell biology experts who use a fluorescent staining technique. This technique consists of coloring target cells with a fluorescent dye to make them distinguishable from other cells. Target cells are then segmented and annotated by recording coordinates of the cell contours within the image.

EVICAN dataset also provides a lighter version where only segmented instances are shown, the background and other cells are blurred. The dataset builders justified this act as it might help build and train models faster. While this is true, it can affect the predictions as the model will not learn from other instances that may help extract more features and distinguish classes.

The following figure visualizes the acquisition pipeline along with the use of the dataset.

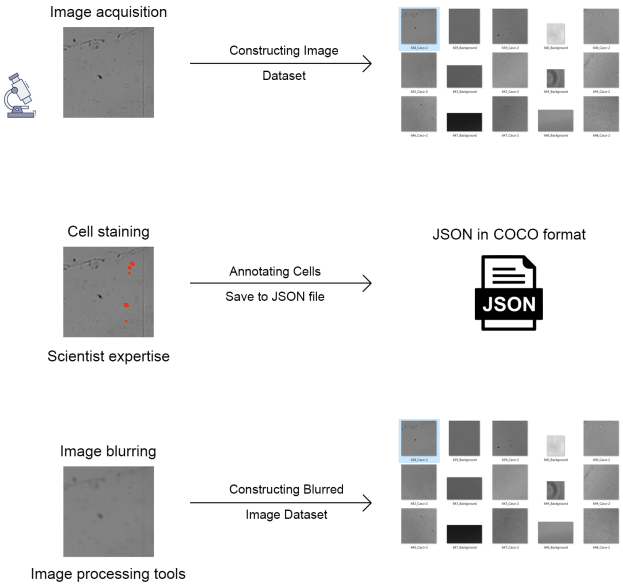
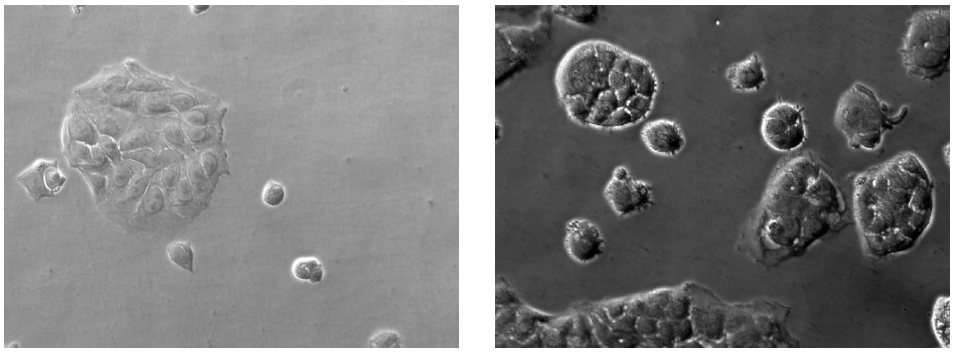


Figure 3: EVICAN dataset construction.

3.3 Custom dataset for colon cells

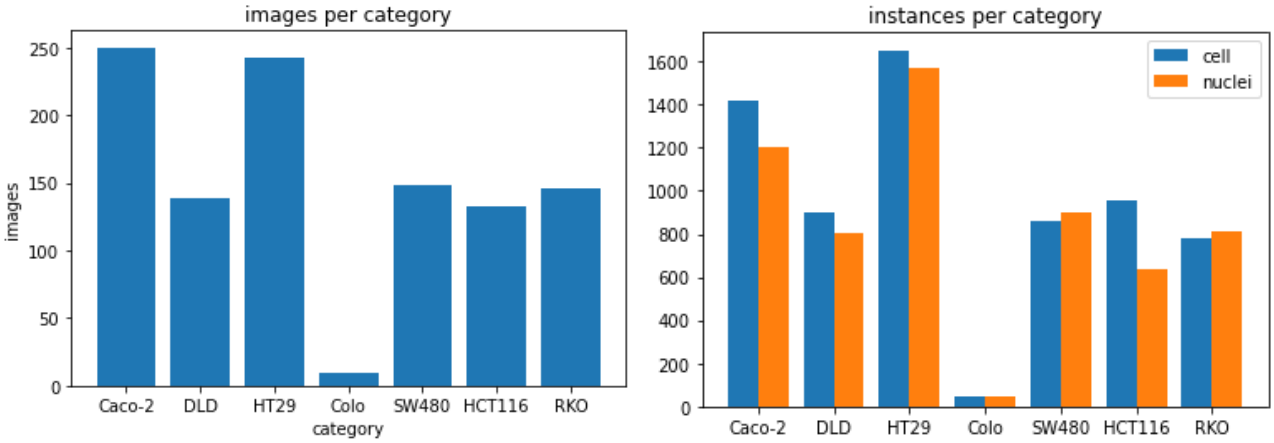
To satisfy the objective of the internship, EVICAN is used to extract a new sub-dataset specific to colon cells. This dataset concerns seven cell lines out of 30 based on their names specified in the EVICAN paper. The extraction operation leads to the following :

- Training dataset: 840 images, 9942 instances (cell+nucleus)
- validation dataset: 207 images, 2422 instances
- Evaluation dataset: 20 images, 218 instances



(a) DLD colon cell. (b) Caco-2 colon cell.

Figure 4: Dataset samples.



(a) Colon cells histogram.

(b) Instances per category histogram.

Figure 5: Colon cell dataset image and instance distribution.

The extracted dataset is also balanced in terms of cell and nuclei instances, except for the "colo" cell line. It is also balanced in terms of contrast variation, cell sizes and image quality.

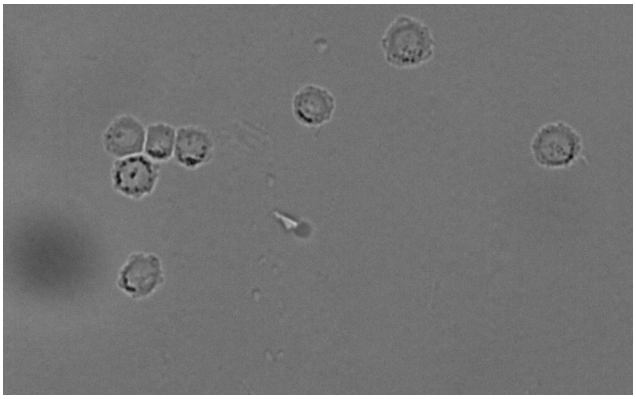
3.4 Difficulty levels

In EVICAN dataset, and therefore in the extracted dataset, the evaluation dataset is divided into three difficulty levels: easy, medium, and difficult. These levels are determined by different characteristics related to imaging quality, brightness, and cell-specific features. The following table summarizes the differences between the three subsets.

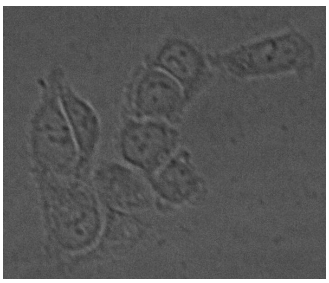
Table 1: Evaluation dataset difficulty criteria.

difficulty / characteristic	Easy	Medium	Difficult
Contrast/ Brightness	Well contrasted image	Low contrast	Low contrast: too bright or too dark
Image size	High resolution	Low resolution	Low resolution
Cell	Separate cells	Cells can be compact	Cells can be compact
Image	Clear background	Background contains artifacts	Background contains noise or artifacts Cells and background are indistinguishable

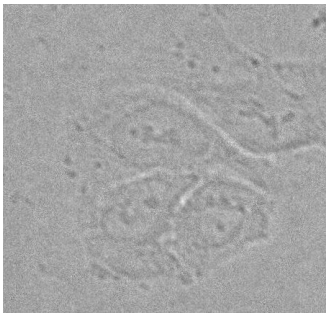
The following figure depicts some of the difficulty levels properties:



(a) Easy sample



(b) medium sample



(c) difficult sample

Figure 6: Difficulty levels samples.

In (a), most cells are separate, the image is well contrasted, and cells and nuclei can be distinguished even with bare eyes. For (b), the image has low resolution, the cells are compact, and have unusual shapes. For (c), the image is too bright, cells are compact, and some parts cannot be differentiated from the background.

4 Modeling

Having the dataset in hand, the next step towards achieving cell segmentation is selecting a technique to perform, optimize and evaluate with well-defined metrics. The choice of such technique can be through reviewing a set of existing model that proved to be efficient for instance segmentation. Some of these methods are cited in A Survey on Instance Segmentation: State of the art [8].
With different approaches to the problem, deep learning models have shown superior results for segmentation. One of the most revolutionary methods is the Mask RCNN architecture, that outscored other techniques on the COCO dataset [9]. That being said, this project will focus on leveraging Mask RCNN for cell image segmentation.

4.1 Mask RCNN

4.1.1 Overview

Mask RCNN (Mask Region-based Convolutional Neural Network) is an object instance segmentation method. Mask RCNN is an extension of previous works on object detection, notably Faster RCNN architecture.
The mask RCNN architecture is a deep architecture composed of different blocks and networks; each performs a specific task to achieve the image segmentation task finally. In this project, Matterport implementation [10] on Python3, Tensorlfow and Keras is employed.

Mask RCNN can be viewed as three main communicating blocks: The backbone, region proposal network (RPN), and the heads. Generally speaking, the backbone plays the role of feature extraction that takes in input images and outputs feature maps.
The backbone is a convolutional neural network (CNN) that is often associated with another feature extractor. In this project, the used backbone is a **ResNet-50** associated with a **Feature Pyramid Network** (FPN).
The region proposal network is a fully convolutional network for object detection within a region. RPN predicts an objectness score (an object is present in that region or not) and a bounding box inside anchors (sliding windows). RPN outputs region proposals which will be crucial for final object detection. Finally, the architecture heads are three parallel branches, responsible for classification, bounding box regression, and mask prediction.
The following figures show an overview of the mask RCNN architecture and the principal used techniques that will be developed in the next section.

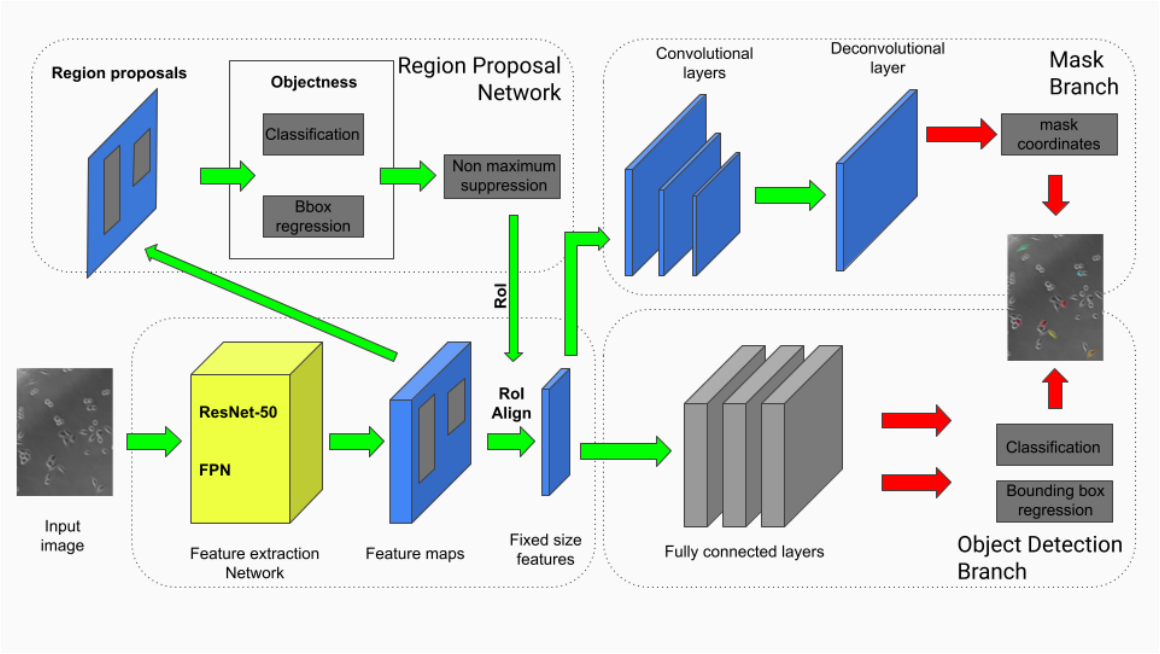


Figure 7: Mask RCNN architecture overview.

4.1.2 Architecture blocks

In this section, the different blocks that make Mask RCNN are explained individually. The relationship between these parts will be depicted while focusing on state-of-the-art techniques and key hyperparameters used in Mask RCNN.

i . **Backbone:**
The backbone, also named feature extraction network, is used to extract features over an entire image. This network can be a CNN, an FPN, or also a combination of both. The latter is used in the Matterport implementation that combines a ResNet-50 and an FPN. This combination proved to be more accurate and fast.

- **ResNet-50**: ResNet or Residual Network [11] is an architecture for CNNs that solves the vanishing gradient problem (a model stops learning due to having an important depth) by introducing a skip connection module.
  ResNet has multiple variants in terms of depth. The choice of a 50-layer depth reduces training time, as many suggest that usually no considerable difference in accuracy is observed compared to deeper variants. A ResNet consists of 5 stages in which multiple residual blocks are used.

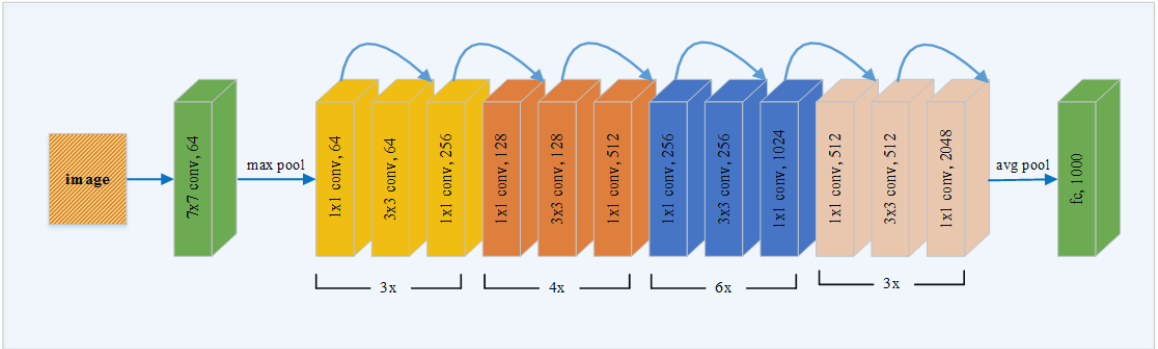


Figure 8: ResNet-50 architecture. [3]

- **Feature Pyramid Network**: It is a feature extractor with two pathways. The bottom-up pathway is for feature extraction from the image, while the top-down pathway takes the extracted features to build higher resolution feature maps with the same semantic value. Both pathways are connected with correspondence of the same level sizes.

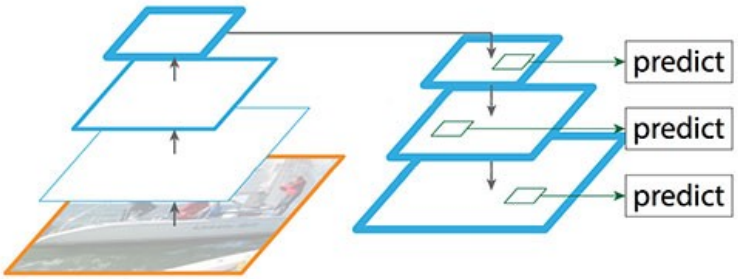


Figure 9: ResNet-50 architecture. [4]

ii . **Region Proposal Network:**
It is a fully convolutional network that takes as input features maps produced by the feature extraction network and applies a sliding window technique over. RPN outputs region proposals or regions of interest (RoI). RPN has a classifier to predict if an object is present within the window and a regressor to predict bounding box coordinates.
RPN generally produces a huge number of proposals for each object with different scores and coordinates. At this point, there is a need to filter out these RoIs and keep the best ones. To do this, non-maximum suppression (NMS) is applied to RoIs corresponding to the same object to keep only one proposal corresponding to the highest objectness.

iii . **RoI align:**
It is an enhancement brought to RoI pooling, present in previous RCNN architectures. Aligning a region of interest consists of performing a pooling operation without quantization. This method is fit for segmentation tasks because it is pixel-to-pixel fit and conserves spatial information. RoI align delivers fixed-size features which serve as input for the architecture heads.

iv . **Heads:**
Mask RCNN heads are two parallel branches that perform threes tasks:

- Object detection branch: Composed of fully connected layers for final classification (cell/nuclei/background) and bounding box regression.
- Mask branch: Uses a fully convolutional network (FCN) for mask prediction. This approach is brought from semantic segmentation, but it performs instance segmentation when combined with the object detection branch.

4.1.3 Key hyperparameters

In order to train and optimize the model on the dataset, different hyperparameters should be understood, defined, and tuned. The following table enumerates key parameters that affect performance the most.

Table 2: List of Mask RCNN key hyperparameters.

Parameter	Architecture block	Role
Learning rate	All	Updates network weights.
Learning momentum	All	Adds consistency to the gradient direction.
Weight decay	All	Penalizes model complexity to avoid overfitting.
Resize mode	RPN	Image input mode : padding, up/down scaling to a defined size
Anchor scales	RPN	Size of RPN windows to detect objects, 5 values corresponding to the 5 levels of the feature exctraction network
Anchor ratio	RPN	Controls shapes of anchors (1 : square, 0.5 and 2 : rectangle)
NMS threshold	RPN	IoU Threshold for an object ,which above overlapping proposals are discarded
Post NMS RoIs	RPN	Maximum number of RoI produced at the output of RPN
FC layers size	Object detection branch	Size of fully connected layers in the object detection layer

4.1.4 Loss function

Mask RCNN loss function is defined as the sum of loss functions of its blocks for each task. The formula is the following:

$$Loss = rpn_bbox_loss + rpn_class_loss + mrcnn_class_loss + mrcnn_bbox_loss + mrcnn_mask_loss$$

where :

- rpn_class_loss = RPN anchor classifier loss
- rpn_bbox_loss = RPN bounding box loss graph
- mrcnn_class_loss = loss for the classifier head of Mask R-CNN
- mrcnn_bbox_loss = loss for Mask R-CNN bounding box refinement
- mrcnn_mask_loss = mask binary cross-entropy loss for the masks head

Each of these loss metrics is the sum of all the loss values calculated individually for each of the regions of interest.

4.2 Transfer learning

As shown in the previous section, Mask RCNN architecture is deep with different blocks to train. Training the whole architecture from scratch is difficult with a relatively small image dataset (840 images), with thousands of parameters to estimate. This task will require significant training time and more computational power and may end up giving poor results. To avoid such problems, transfer learning can be an effective solution.

Transfer learning is a machine learning technique where a model trained on one task is repurposed on a second related task [12]. For further explanation, transfer learning is when the result of training a model on a general dataset is used to train another model on a more specific dataset. The features learned from the first task are more general and facilitate adapting shallow layers of a model.

One method of transfer learning is the pre-trained model approach that consists of three steps which are:

- **Select source model:** Choose weights that correspond to the architecture from one of the available pre-trained weights published by researchers.
- **Exclude unnecessary weights:** Discard top layer weights because their number differs from one dataset to another depending on the number of classes in the dataset.
- **Tuning model:** The pre-trained weights can instantiate the model weights as a start. Then, the model can be totally retrained on the new dataset or partially.
  Training a model partially consists of selecting parts (blocks) to train while freezing the other parts. The typical routine is to freeze shallow layers of the architecture with less semantic value, train deeper layers with more semantic value, and contribute more to the classification/detection task. Untrainable layers can be gradually decreased until the totality of the network is trained.

To apply these steps on the mask RCNN architecture, their are employed as follows :

- Pre-trained weights for mask RCNN with a ResNet-50 backbone were used. These weights are trained on the COCO dataset.
- The following layers were excluded before loading the pre-trained weights: "mrcnn class logits", "mrcnn bbox fc", "mrcnn bbox", "mrcnn mask". These weights correspond to the final output of the classification, localization, and segmentation of cells, as the pre-trained weights correspond to the 80 classes of the COCO dataset, unlike the two classes of EVICAN.
- Tuning mask RCNN weights has different options for selecting layers to train using the following keywords:
  - heads : train RPN + FPN + detection and mask branches.
  - 5+ or 4+ : train ResNet block 4 or 5 and above.
  - all : train all architecture layers.

4.3 Evaluation Metrics

To evaluate a model trained on the dataset, mean average precision metric (mAP) was chosen. This metric is the standard choice for an instance segmentation task. Mean average precision is calculated at different Jaccard index thresholds.

4.3.1 Jaccard index

The Jaccard index, also known as Intersection over Union (IoU), is a statistic used to measure similarity between samples. It is defined as the intersection divided by the union of finite samples. The mathematical formula is the following:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

It can be also visually represented as depicted in Figure 10:

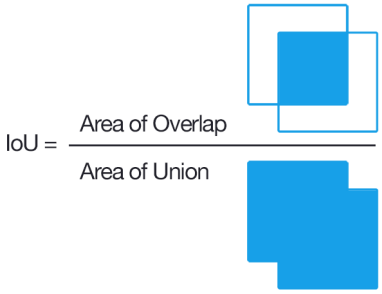


Figure 10: Illustration of Jaccard index.

In this project, the Jaccard index calculates the average precision and recall at an IoU threshold value. This approach guarantees the evaluation of confident predictions that bypass the threshold. An ideal value is 1, where the two samples are identical. A value close to 0 means a bad prediction. Values over 0.5 are generally taken into consideration.

4.3.2 Precision and recall

Precision is a metric to evaluate classification. Precision is the ratio of true positive and total predicted positives (true positives and false positives) for a certain class. The formula is such as :

$$Precision = \frac{TP}{TP + FP}$$

Recall is the true positive rate; it evaluates how well, ground truth objects are being correctly detected. It is given by the formula :

$$Recall = \frac{TP}{TP + FN}$$

In object detection or image segmentation context, a detection is considered a true positive (TP) if the classification is correct and the IoU value between the ground truth mask/bounding-box and the predicted one is superior to a defined threshold. On the other hand, a detection is considered a false positive (FP) if the IoU is inferior to the threshold or the detection is a duplicate with a lower IoU value. Finally, a false negative (FN) corresponds either to no detection or if the IoU is superior to the threshold, but the object is misclassified.

4.3.3 Mean average precision

Mean Average Precision is a metric widely used in object detection and segmentation tasks. It is used to evaluate the classification of detected objects at an IoU threshold, which evaluates object localization.
Average precision is defined as the area under the Precision-Recall curve for each class. The

obtained values are averaged over the number of classes for multiple classes, giving the mean Average Precision.

4.3.4 Metric interpretation

- High recall but low precision implies that all ground truth cells/nuclei have been detected, but most detections are incorrect (many false positives).
- Low recall but high precision implies that all predicted boxes are correct, but most ground truth cells/nuclei have been missed (many false negatives).
- High precision and high recall, the ideal detector has most ground truth cells/nuclei detected correctly.

A high mAP value (close to 1) is the result of a high area under the curve. That represents both high precision and high recall.

5 Training environment

In this project, both training and inference were performed using Google Colab on a GPU accelerated environment. The allocated GPU by Colab is a 12GB NVIDIA Tesla K80, along with 16GB of RAM. This allowed an average of 14 minutes for an epoch to train on the dataset and 17 minutes on the augmented dataset.
For the software components, the model is built and trained using mainly Python 3.7, Tensorflow-gpu 1.15 and Keras 2.1.6. Other python packages were used for different purposes and are enumerated in the 'requirement.txt' file within the repository.

It should be noted that the Matterport implementation was not maintained since 2019 and required an effort to fix conflicts and errors in packages and code. This effort led to updating the 'requirement.txt' file and some code lines to get the Mask RCNN library run.
Other used tools in the workflow are:

- Tensorborad: a tool used to log the training process and enables pause and continue training. It is also used to plot and debug learning curves.
- ImgAug : a library for building image augmentation pipelines. This library is compatible with Tensorflow and Keras which facilitates the pipeline integration within the model. ImgAug provides multiple ready-to-use transformation with the ability to customize different parameters. It also offers building image augmentation with random or probabilistic approaches.
- ImageJ : an open-source tool for image processing. This tool helped studying dataset images especially through histograms. Also, it helped choosing image transformations that do not affect image quality thanks to its large arsenal of image processing techniques.

6 Training and Results

6.1 Base model

The base model is the first model to be trained using hyperparameters default values as proposed in the Matterport implementation.

6.1.1 Training strategy

As previously mentioned in Transfer learning section, multi-stage training is a key technique in transfer learning. Each time, a training strategy will be defined concerning architecture blocs to train or hyperparameters to use. For the base model the strategy is the following:

- **Stage 1:**
  - Train network heads for 20 epochs with learning rate = 0.001.
  - Train network heads for 20 epochs with learning rate decreased to 50% .

  This stage aims to adapt network heads for the prediction task on cell images rather than objects present in COCO dataset.

- **Stage 2:**
  - Fine-tune ResNet 4th block and above layers (ResNet block 4, block 5 and network heads) for 15 epochs.

  Deeper ResNet blocks (4th and 5th) have greater semantic value than shallow blocks, so fine-tuning their weights will make them better suited for prediction tasks on the dataset.

The employed hyperparameters are kept to default values defined in the configuration class in the repository, except for the backbone which was altered from ResNet-101 to ResNet-50 to save training time. This choice was backed by the experience of other users who noted that this change does not affect performances noticeably.

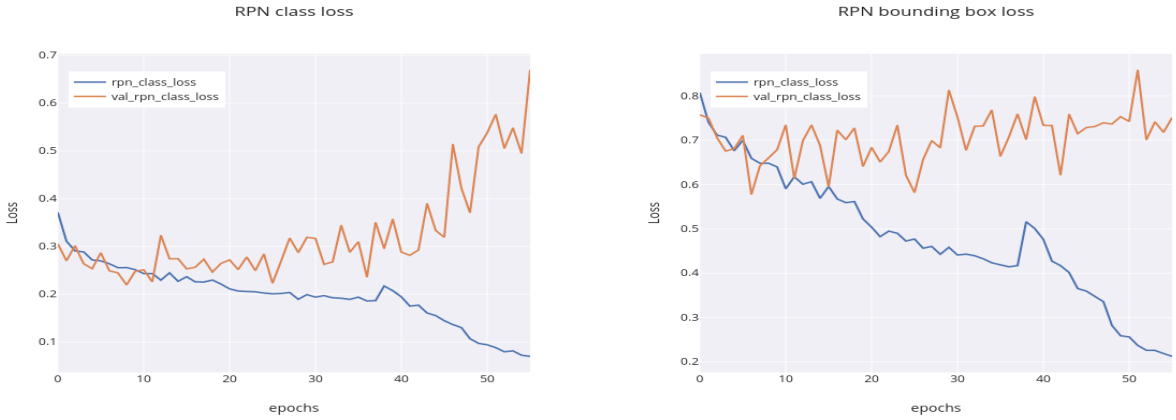
Table 3: Default hyperparameters.

Parameter	Value
Learning rate	0.001
Learning momentum	0.9
Weight decay	0.0001
Anchor scales	[32, 64, 128, 256, 512]
Anchor ratio	[0.5, 1 , 2]

To track the learning process, learning curves for the whole model and for the region proposal network, as it contributed the most the total loss, were plotted.



Figure 11: Base model learning curve.



(a) RPN class loss.

(b) RPN bounding box loss.

Figure 12: Region proposal network learning curve.

The first stage of training demonstrates a gradual decrease in training loss while keeping stable values for validation loss. Moving to the second stage, a divergence between training loss and validation loss is noticed. This same divergence is observed in RPN learning curves which contribute the most in the total loss.

6.1.2 Results

Depending on the metrics defined in Evaluation Metrics, the model is evaluated using mean Average Precision at IoU thresholds equals to 0.5 and 0.75 for three different difficulty levels. The results are the following:

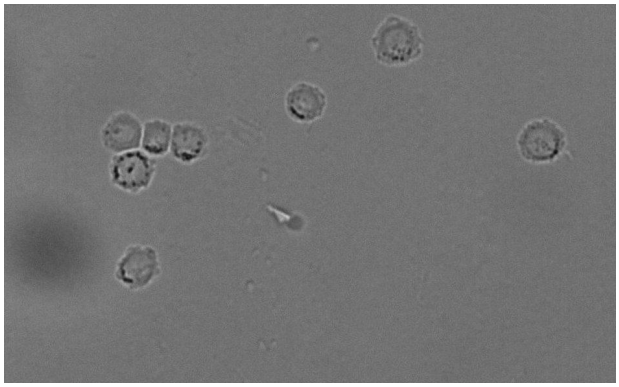
Table 4: Base model results.

metric \ difficulty	easy	medium	difficult
mAP @IoU=0.5	0.645	0.012	0.004
mAP @IoU=0.75	0.314	0.01	0.024

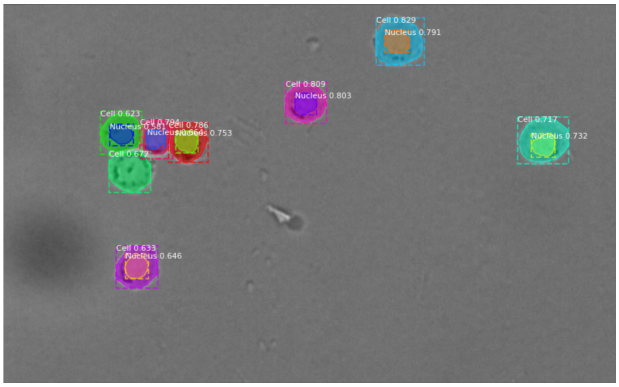
The base model shows mediocre results for the easy evaluation set while not showing any performance on other difficulty levels.
This poor performance reflects the fact that the model is overfitting, based on the learning curve divergence. Two hypothesis are developed : Either the model is overfitting while training deeper layers (2nd stage) or the region proposal network is not detecting objects well to generate good RoIs. To confirm or deny one of the hypothesis, deeper layers will be freezed and RPN hyperparameters will be tweaked.

6.1.3 Visualization

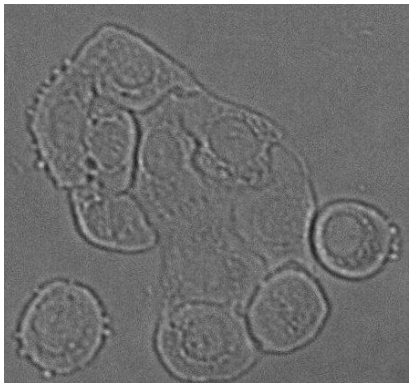
To visualize and assess the performance, one image out of each difficulty set is chosen randomly and evaluated. These images will serve as well for the next models as a tool for visualization.



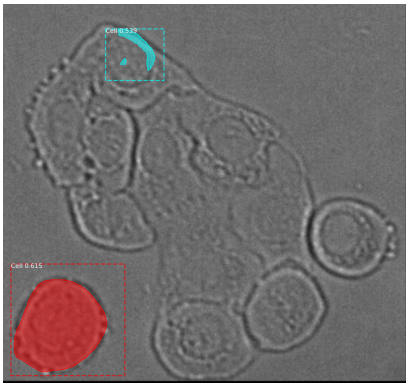
(a) Easy simple



(b) Easy simple segmentation



(c) Medium sample



(d) Medium sample segmentation



(e) Difficult sample



(f) Difficult sample segmentation

Figure 13: Visualization of random segmented samples.

In sample (a), cellular membranes are highlighted and are easy to detect even when cells are compact. Nuclei inside cells have clear edges. The background is clear and no significant artifacts are present. In sample (c), cells are compact, membranes are too thin to show a frontier between cells that also have irregular shapes. Sample (e) is blurry and cells do not keep a uniform shape. These visualization also confirm the obtained results. While, all cells and nuclei are detected and segmented on the easy sample, the model barely detected cells in the other two samples.

6.2 Hyperparameter tuning

To optimize the model, different parameters must be tuned along with the training strategy. In this part, the choice of hyperparameters values is based on previous hypothesis in the previous section and also on the community experience with the model.

6.2.1 Tuning strategy

First of all, the training strategy is now limited to the network heads, as the model previously tends to overfit when training the backbone layers. In this part the heads will be trained for 30 epochs with a learning rate initially set to 0.001 and then for an additional 30 epochs with a learning rate cut to half. Second, weight decay is increased to penalize the model complexity more. Finally, RPN anchor sizes are reduced, as the default values are too high to fit for small size cells and especially nuclei.

Table 5: Tuned hyperparameters.

Parameter	Value
Learning rate	0.001
Learning momentum	0.9
Weight decay	0.001
Anchor scales	[16, 32, 64, 128, 256] and [8,16, 32, 64, 128] (to study the best fit size for cell and nuclei at multiple scales.)

Monitoring the learning curve, the resulting training and validation loss do not diverge.

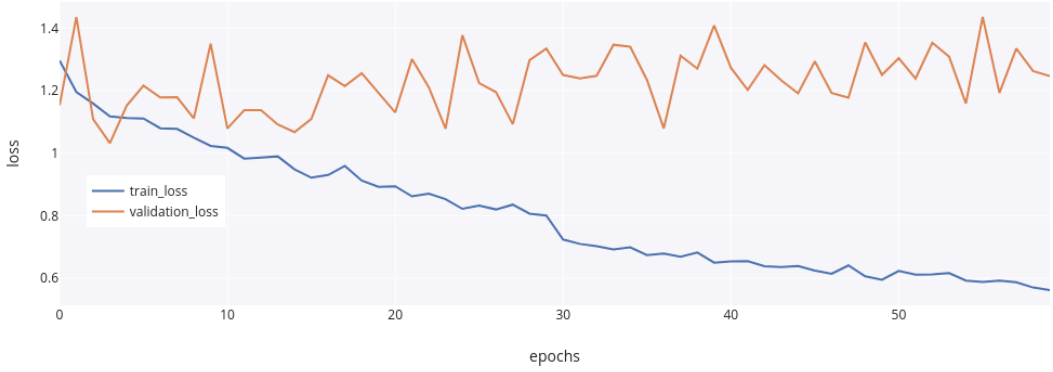
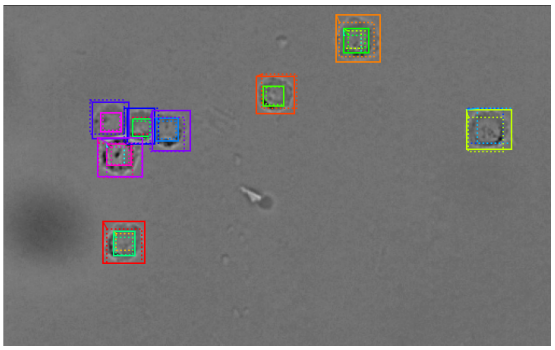
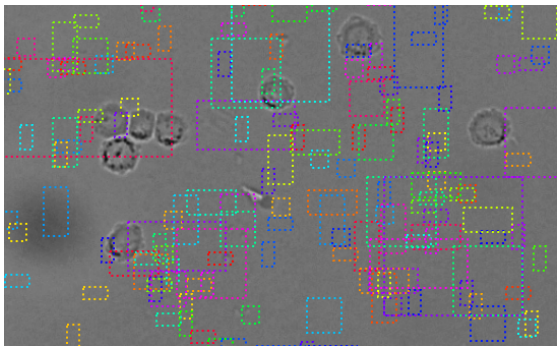


Figure 14: Tuned model learning curve.

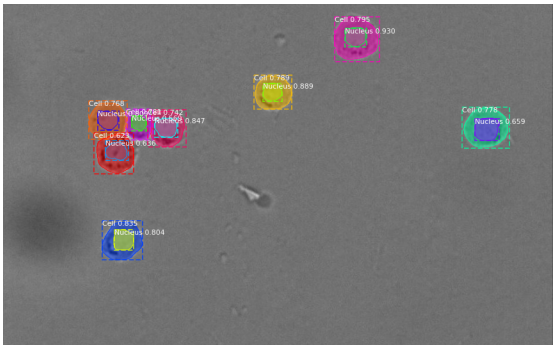
The final step is to choose which anchor scales are suited for the dataset. To do this, a visual representation of rpn anchors is put to test. The represented anchors are random and do not represent the totality of them.



(a) Positive anchors

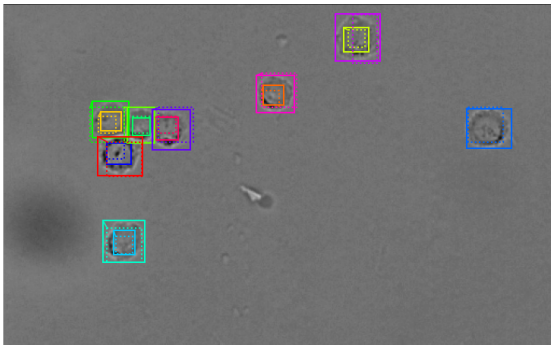


(b) Negative anchors

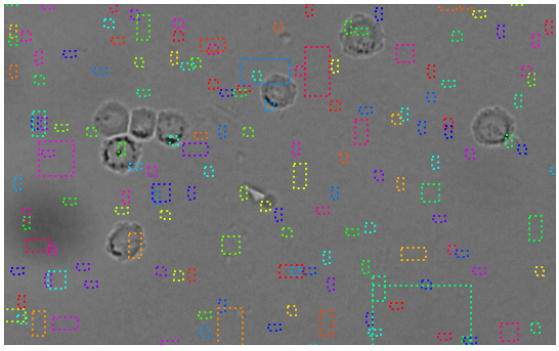


(c) Final segmentation

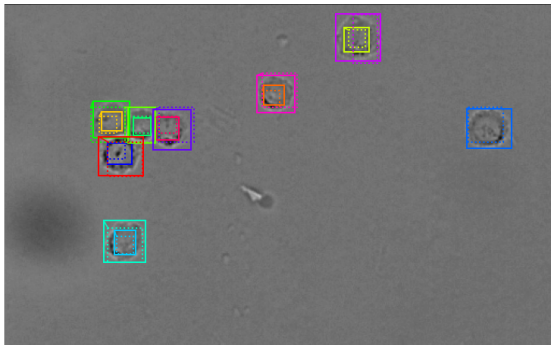
Figure 15: RPN anchors = [8, 16, 32, 64, 128].



(a) Positive anchors



(b) Negative anchors



(c) Final segmentation

Figure 16: RPN anchors = [16, 32, 64, 128, 256].

In figure 15, cells and nucleus detected with good to high probabilities. On the other hand, nucleus are not present at the output although there are positive anchors meaning they have low objectness scores that don't qualify them as ROIs. Nucleus are smaller objects than cells thus the need of smaller anchors to detect them. Therefore, the decision is to opt for anchor scales [8,16, 32, 64, 128].

6.2.2 Results

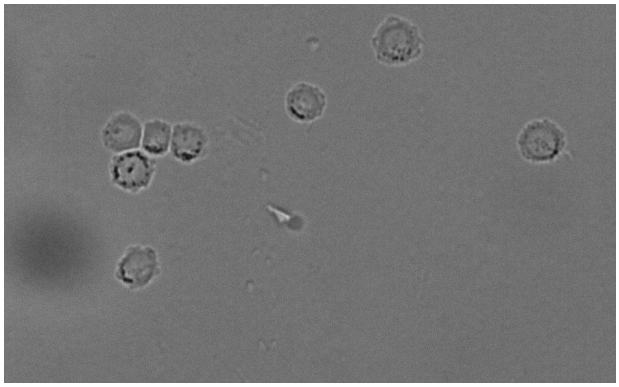
The tuned model shows a significant improvement on the three evaluation difficulty levels, especially it can now detect cells on medium and hard images even so the performance is still poor.

Table 6: Optimized model results.

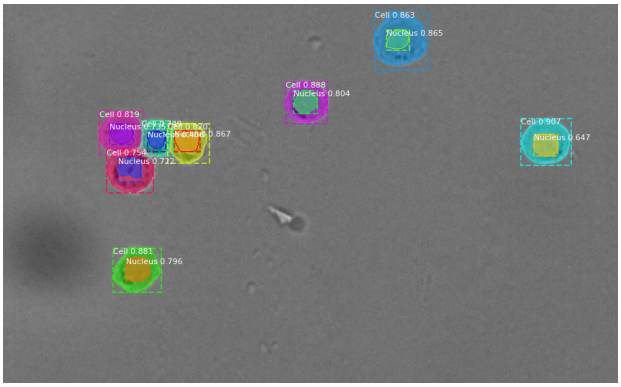
metric \ difficulty	easy	medium	difficult
mAP @IoU=0.5	0.819	0.178	0.183
mAP @IoU=0.75	0.445	0.023	0.006

6.2.3 Visualization

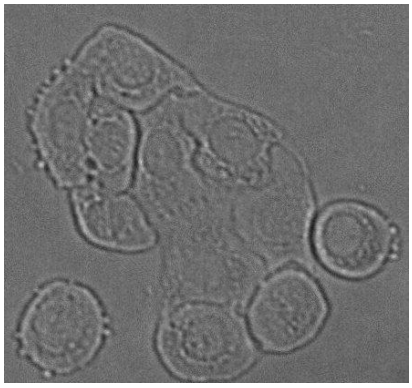
The same tested images are visualized for the new model.



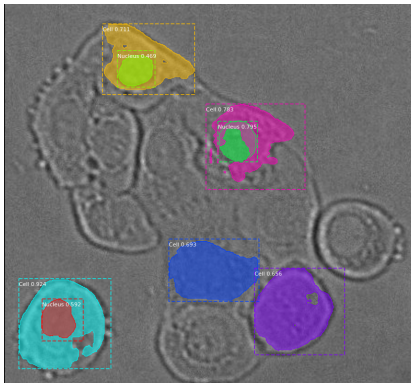
(a) Easy simple



(b) Easy simple segmentation



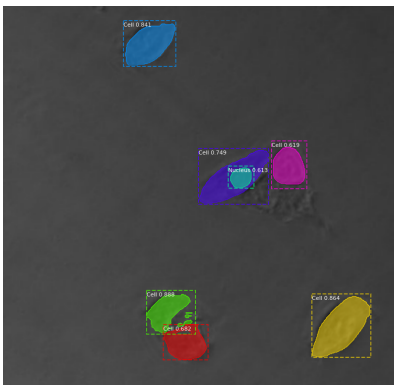
(c) Medium sample



(d) Medium sample segmentation



(e) Difficult sample



(f) Difficult sample segmentation

Figure 17: Visualization of random segmented samples.

Segmentation of sample (a) has a higher confidence score for instances. Sample (c) and sample (e) show better results than previous model.

6.3 Image augmentation

Giving the limited size of the dataset to train such a deep learning model, an image augmentation pipeline is deployed. Image augmentation consists of applying different operations on the image dataset in order to obtain more training samples and make the model more robust against more general images.

6.3.1 Techniques

To perform image augmentation on the extracted EVICAN dataset, one must be careful with used techniques, as cell images can be sensible to some transformation such as scaling that can change the nature of the image properties. Candidate transformations are split to two classes:

- **Position variant:** It consists of horizontal and vertical flips and rotations.
- **Position invariant:** It consists of a manipulation of the image color settings such as contrast, brightness or applying filters such as blurring.
  In this project, the used technique is Contrast Limited Adaptive Histogram Equalization **(CLAHE)**. CLAHE is an algorithm used to enhance images contrast. CLAHE operates on small regions in the image, called tiles, rather than the entire image which is its the key for its good performance on bad contrasted images. For each tile, the algorithm calculates local histogram, equalize it while taking care of the resulting noise. Applied on EVICAN, CLAHE improved significantly dark and bright images, revealing some cells and features that were not visible or clear on the original images.

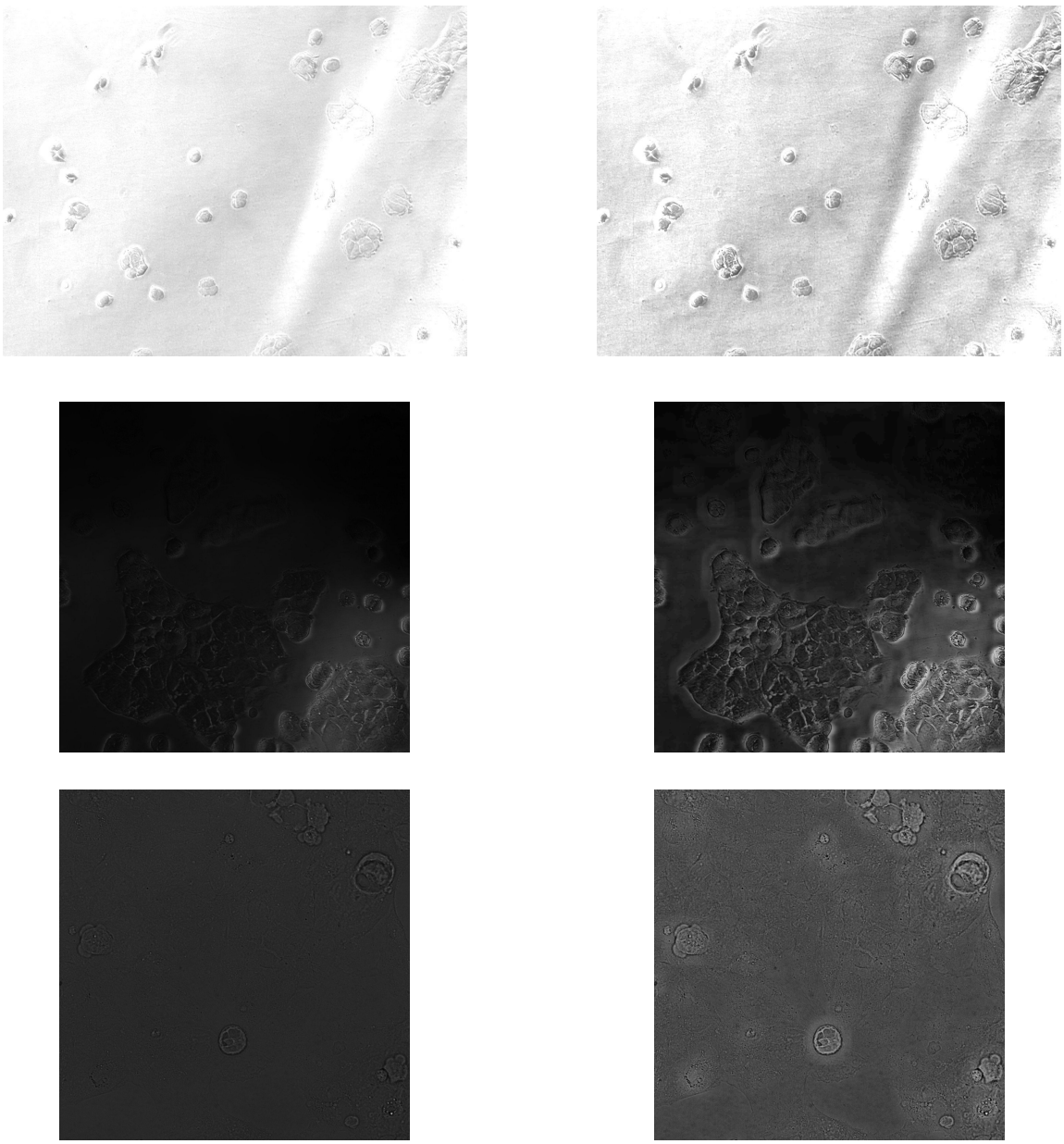


Figure 18: Comparison between original samples (left) and CLAHE enhancement (right).

6.3.2 Results

To train the model with image augmentation, the last model training strategy and hyperparameters were preserved. Training dataset size is set to double (∼1600 images). The results are the following:

Table 7: Model results with image augmentation.

difficulty / metric	easy	medium	difficult
mAP @IoU=0.5	0.527	0.051	0.099
mAP @IoU=0.75	0.333	0.01	0.022

Performance on the evaluation set seems to fall. This can be due to different reasons: either

the training strategy is not enough to train on augmented data or the applied transformations are not very suited to the dataset.

7 Conclusion

The aim of this project was to leverage existing assets and adapt them in order to develop a solution for colon cell segmentation, that can identify and segment cancerous cells.

Going through the EVICAN dataset, studying its properties and understanding how the difficulty levels can affect a model's assessment was an important task. Medium and hard difficulty levels correspond to extreme cases where the imaging conditions or the image processing are bad, which is a rare case in the present regarding the development of such technologies.

The modeling phase, demonstrated that state-of-the-art architectures must be adapted and tuned in order to fit new datasets. The tuning process requires understanding how the employed architecture works at its different levels and how will it behave if a certain parameter is modified. This was a preparation phase for training the model on the data.

Evaluating the model and interpreting the results serve as a feedback to the network, as it should be optimized based on previous tests to debug where problems lay and try to fix them.

Finally, this project still has the capacity to improve even though its performance is acceptable on the easy set that can be a starting point to test it on more general images from outside EVICAN.

References

[1] Ben Dickson. New deep learning model brings image segmentation to edge devices. https://bdtechtalks.com/2021/05/07/attendseg-deep-learning-edge-semantic-segmentation/, 2021.

[2] Pulkit Sharma. Computer vision tutorial: A step-by-step introduction to image segmentation techniques. https://analyticsvidhya.com/blog/2019/04/introduction-image-segmentation-techniques-python/, 2019.

[3] TMuhammed Taloa. Convolutional neural networks for multi-class histopathology image classification, 2019.

[4] Tsung-Yi Lin, Piotr Dollár, Ross Girshick, Kaiming He, Bharath Hariharan, and Serge Belongie. Feature pyramid networks for object detection, 2017.

[5] Pavan Vadapalli. Image segmentation techniques. https://upgrad.com/blog/image-segmentation-techniques/, 2019.

[6] Seiya Fujita and Xian-Hua Han. Cell detection and segmentation in microscopy images with improved mask r-cnn. In *Proceedings of the Asian Conference on Computer Vision (ACCV) Workshops*, November 2020.

[7] Feixiao Long. Microscopy cell nuclei segmentation with enhanced u-net. *BMC Bioinformatics*, 21, 01 2020. doi: 10.1186/s12859-019-3332-1.

[8] Abdul Mueed Hafiz and Ghulam Mohiuddin Bhat. A survey on instance segmentation: State of the art. *CoRR*, abs/2007.00047, 2020. URL https://arxiv.org/abs/2007.00047.

[9] Zhaojin Huang, Lichao Huang, Yongchao Gong, Chang Huang, and Xinggang Wang. Mask scoring r-cnn, 2019.

[10] Waleed Abdulla. Mask r-cnn for object detection and instance segmentation on keras and tensorflow. https://github.com/matterport/Mask_RCNN, 2017.

[11] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition, 2015.

[12] Jason Brownlee. A gentle introduction to transfer learning for deep learning. https://machinelearningmastery.com/transfer-learning-for-deep-learning/, 2017.